HDOC Day-51

Name	Gurveer Singh Cheema
SAP ID	590032925
Batch	15

Problem Statement

Write a C program that reads the size and elements of a variable-length integer array **arr[]**. Pass the array to a function named **next_greater()** using its address. Use pointers to pass the array between the calling function and **next_greater()**.

For each array element, search only the elements to its right and display the nearest element that is greater than the current element. Display **-1** if no greater element exists. Use nested loops and print the results in array order, separated by commas.

C Program

```
#include <stdio.h>

void next_greater(int *arr, int size)
{
    int i, j, found;

    for (i = 0; i < size; i++)
    {
        found = -1;

        for (j = i + 1; j < size; j++)
        {
            if (*(arr + j) > *(arr + i))
            {
                found = *(arr + j);
                break;
            }
        }

        printf("%d", found);
        if (i < size - 1)
            printf(", ");
    }
    printf("\n");
}

int main()
{
    int size, i;

    printf("Enter size of array: ");
    scanf("%d", &size);

    int arr[size];

    printf("Enter array elements: ");
    for (i = 0; i < size; i++)
        scanf("%d", arr + i);

    printf("Next greater elements: ");
    next_greater(arr, size);

    return 0;
}
```

Explanation

The array address is passed to **next_greater()** through the pointer **int *arr**. For each element, the inner loop starts from the next position and searches toward the right. The first element greater than the current element is the nearest next greater element, so it is stored and the loop stops using **break**. If no greater element is found, **-1** is printed.

Sample Outputs

```
Example 1:  4 5 2 25     ->  5, 25, 25, -1
Example 2:  13 7 6 12    ->  -1, 12, 12, -1
Example 3:  1 2 3 2 4    ->  2, 3, 4, 4, -1
Example 4:  1 2 3 4 5    ->  2, 3, 4, 5, -1
```

Result

Thus, the program successfully finds the nearest next greater element on the right for every array element using pointers and nested loops.